

SGITU – Système de Gestion Intelligente des Transports Urbains

Approche : Architecture Microservices + UML + DevOps

1. Objectif global du projet

Concevoir et modéliser un système **SGITU** basé sur une **architecture microservices**, en :

- Appliquant les concepts de **modélisation UML**
- Développant des **microservices REST**
- Intégrant des outils modernes (**conteneurs, orchestration, sécurité**)

2. Organisation par groupes

Chaque groupe prend en charge **un sous-système (microservices métier)**.

Répartition proposée (10 groupes)

<i>Groupe</i>	<i>Sous-système</i>	<i>Description</i>
G1	Billetterie dématérialisée	Achat, validation des tickets
G2	Gestion des abonnements	Souscription, renouvellement
G3	Gestion des utilisateurs	Comptes, profils
G4	Coordination des transports	Synchronisation bus/tram
G5	Notifications	SMS, email, alertes
G6	Paieement	Transactions, facturation
G7	Suivi des véhicules	Localisation temps réel (IoT)
G8	Analyse & données	Statistiques, Big Data
G9	Gestion des incidents	Pannes, alertes
G10	API Gateway & Sécurité	Authentification + routage

3. Architecture globale attendue

Chaque groupe doit s'intégrer dans :

- Une architecture **microservices distribuée**
- Communication via **API REST (HTTP)**
- Base de données **indépendante par service**

Vue globale :

- Frontend (optionnel)
- API Gateway
- Microservices
- Base de données
- Conteneurs Docker

4. Travail demandé par groupe

Partie 1 – Analyse & Modélisation (UML)

Objectif : Structurer le système avant implémentation

À produire

- Diagramme de cas d'utilisation
- Diagramme de classes
- Diagramme de séquence
- Diagramme de composants (microservices)

Outils

- Draw.io
- Lucidchart

Partie 2 – Conception Microservices

Objectif : Définir le service

À produire

- Liste des endpoints REST
- Verbes HTTP (GET, POST, PUT, DELETE)
- Gestion des erreurs (codes HTTP)
- Documentation API

Outils

- Swagger

Partie 3 – Implémentation

Objectif : Développer le microservice

À produire

- Microservice fonctionnel
- API REST testable

Outils

- Spring Boot
- Spring Cloud

Partie 4 – Tests

À produire

- Tests via Postman
- Scénarios d'utilisation

Outils

- Postman

Partie 5 – Conteneurisation

Objectif : Déployer le microservice

À produire

- Dockerfile
- Image Docker

Outils

- Docker

Partie 6 – Orchestration

À produire

- docker-compose.yml
- Intégration avec autres services

Outils

- Docker Compose

Partie 7 – Sécurité (pour tous + G10 en lead)

À produire

- Authentification (JWT)

- Sécurisation des endpoints

Outils

- Spring Security

5. Interconnexion entre groupes

Chaque groupe doit :

- Consommer au moins **1 microservice externe**
- Exposer une API utilisée par un autre groupe

Exemple :

- G1 (Billetterie) → utilise G6 (Paiement)
- G5 (Notifications) → utilisé par tous

6. Livrables globaux

Par groupe

- Rapport (10–15 pages)
- Diagrammes UML
- Code source
- API documentée
- Dockerfile

Global (classe)

- Architecture complète SGITU
- Démo intégrée (tous les services)

7. Bonus (groupes avancés)

- Implémentation avec Quarkus ou Micronaut
- Déploiement sur Kubernetes
- Intégration IoT (capteurs véhicules)